

Automation QA Engineer

Practical Assessment — Proton Mail Web Application

Candidate	Date / Submission	Estimated Time
		4–6 hours
Role: Automation QA Tester	Application: Proton Mail Web	Web automation focus

Assessment Overview

This assessment evaluates how you approach software quality as an Automation QA Engineer — not simply how quickly you can write UI scripts. You will explore a real web application, identify high-value scenarios, design a sensible test strategy, and automate a maintainable regression suite.

Primary application

Proton Mail Web — <https://proton.me/mail>

You may use the live application and Proton’s public documentation as supporting references. The live application is the system under test; document any behavior you observe that differs from documentation.

What we are looking for

- Strong test-case design and prioritization, including positive, negative, boundary, and state-transition scenarios.
- Practical judgment about what should be automated, what should remain manual/exploratory, and why.
- Clean, maintainable automation code with reusable abstractions, reliable synchronization, and meaningful assertions.
- Ability to test workflows end-to-end across multiple application states and, where appropriate, multiple user accounts.
- Good debugging discipline: clear failure evidence, useful reporting, and a reproducible test structure.

1. Functional Scope

Use the following business-level areas as your minimum scope. You are responsible for deriving the detailed test cases, priorities, test data, and automation strategy.

Area	What to cover
Authentication & session	Login, validation, logout, session state, refresh/back navigation, and relevant security-related behavior.
Compose & send	Recipient handling, subject/body, CC/BCC, validation, send flow, and post-send verification.

Area	What to cover
Drafts	Auto-save/state persistence, reopen/edit, discard/delete, refresh behavior, and transition from draft to sent.
Inbox & message actions	Open/read/unread, star, archive, delete/trash, restore, bulk actions, and resulting message state.
Folders & labels	Create/manage, move messages, apply/remove labels, and verify resulting organization/state.
Search	Keyword, sender/recipient, folder, date range, no-result behavior, clear/reset, and at least one advanced-search scenario.
Filters / automation rules	Create, enable/disable, edit, delete, matching vs non-matching messages, and rule behavior.
Attachments	Upload, remove, send, receive/download, filenames, and draft persistence with attachments.
Asynchronous workflows	Include at least one timing/state-dependent flow, such as undo-send or scheduled sending.

Recommended advanced coverage

- Scheduled email: create, verify scheduled state, edit/cancel, and discuss how you would test time-dependent behavior without creating long-running tests.
- Undo Send: verify the transient undo state and the final send state; avoid arbitrary fixed sleeps where reliable synchronization is possible.
- Password-protected email or other security/privacy-sensitive workflows, where your environment permits safe testing.
- Cross-browser execution, parallelization, API-assisted test-data setup, or CI execution as bonus areas.

Important

Do not treat this list as a checklist to click through. We are evaluating your ability to decide what matters, identify risk, and build a focused automation suite.

2. Test Strategy & Test Cases

Before writing automation, prepare a concise test strategy. Your test cases should show how you think about risk and coverage.

Deliverable	Expected content
Test strategy	Scope, assumptions, risks, priorities, test levels, automation approach, and key exclusions.
Test cases	Clear preconditions, steps, test data, expected result, priority, and automation classification.
Coverage rationale	Explain why the selected automated scenarios provide good regression value.
Non-automated scenarios	Identify important scenarios you would keep manual/exploratory and explain why.

Prioritization

- P0 — Critical: core functionality or a failure that blocks a major user journey.
- P1 — High: significant functionality with high business/user impact.
- P2 — Medium: useful coverage with lower immediate risk.
- P3 — Low: edge, cosmetic, or low-risk scenarios.

Automation classification

- Automate — stable, repeatable, high-value regression coverage.
- Manual / Exploratory — scenarios where human observation, investigation, or changing behavior provides more value.
- Not worth automating — low-value, highly brittle, redundant, or expensive-to-maintain checks.

3. Automation Implementation

Implement a small but production-minded automation suite. We would rather see 15 reliable tests than 60 brittle scripts.

Minimum automation requirements

- Automate approximately 15–20 high-value scenarios across multiple scope areas.
- Use a clear project structure with reusable page/component abstractions or an equivalent design.
- Use meaningful assertions that validate application state, not only element visibility/clickability.
- Use robust synchronization/waiting strategies. Avoid unnecessary hard-coded sleeps.
- Parameterize or externalize repeatable test data where appropriate.
- Capture useful failure evidence such as screenshots, traces, videos, logs, or equivalent artifacts.
- Provide a readable test report or test-result output.
- Make the suite straightforward for another QA engineer to install, run, debug, and extend.

End-to-end expectation

Two-account workflow (recommended)

Where practical, use two test mailboxes to validate a real flow such as Sender → Send → Receiver → Verify message. You may use API/setup helpers or other safe mechanisms to control test data if you explain them in the README.

Handling asynchronous behavior

At least one automated scenario should involve a timing- or state-dependent workflow. Your implementation should demonstrate how you deal with eventual UI state changes without making tests unnecessarily slow or flaky.

Optional / bonus engineering practices

- Cross-browser configuration.

- Parallel test execution.
- CI workflow (GitHub Actions or equivalent).
- Environment-based configuration and secure secrets handling.
- API or direct-data helpers for reliable setup/cleanup where appropriate.
- Retry strategy used carefully and only where justified, rather than masking genuine defects.

4. Required Scenarios to Demonstrate

Your final suite must contain coverage for each of the following categories. You decide the exact test scenarios within each category.

Category	Minimum expectation
Authentication	At least 2 automated scenarios, including a negative case.
Compose / send	At least 3 automated scenarios, including a validation/negative case.
Draft / persistence	At least 2 automated scenarios.
Inbox / organization	At least 3 automated scenarios across message actions.
Search	At least 2 automated scenarios, including one advanced or multi-condition case.
Filters	At least 1 automated scenario.
Async behavior	At least 1 automated scenario (e.g., undo-send or scheduling).
Attachments	At least 1 automated scenario.

5. Defect Reporting Challenge

While exploring the application, identify any genuine defects, inconsistencies, usability issues, or automation-relevant risks you encounter. Do not manufacture bugs. Distinguish clearly between a defect and an assumption/limitation.

For each defect, provide

Field	What to include
Title	Clear, concise defect summary.
Environment	Browser, OS, viewport, build/date if known.
Severity / priority	Your reasoning, not only a label.
Preconditions	Required account/data/state.
Steps to reproduce	Minimal, deterministic steps.
Expected vs actual	What should happen vs what happened.
Evidence	Screenshot, video, trace, console/log evidence where relevant.
Reproducibility	Always / often / intermittent, with observations.

6. Automation Quality Review

Include a short section in your README answering the questions below. The quality of the reasoning matters more than the length of the answers.

1. Which tests did you deliberately choose NOT to automate, and why?
2. Which 5 tests would you run on every pull request? Which would you run nightly? Explain the trade-off.
3. How would you diagnose a test that passes locally but fails intermittently in CI?
4. How would you reduce test-suite flakiness if the UI changes frequently?
5. How would you scale this suite from 20 tests to 200+ tests while keeping maintenance manageable?

What a strong answer looks like

We are looking for practical engineering judgment: risk-based prioritization, deterministic tests, strong assertions, clean abstractions, useful diagnostics, and clear trade-offs.

7. Submission Requirements

Required item	Submission expectation
Source code	Complete runnable automation project.
README.md	Setup, prerequisites, configuration, account/test-data assumptions, commands, architecture, and troubleshooting notes.
Test cases	Spreadsheet/CSV/Markdown/Docs format is acceptable. Include priority and automation classification.
Test report	Execution results for the submitted suite, including failures if any.
Defect report	Any genuine defects or meaningful findings discovered during testing.
Architecture note	Brief explanation of framework structure and important design decisions.
Video walkthrough	A 4-5 minute video explaining your work.

Repository structure

Keep the project organized and include the following at minimum:

```
README.md
tests/
pages/ or equivalent abstraction layer
config / test-data / fixtures (as appropriate)
```

reports/ or documented command to generate reports
 package/dependency configuration
 optional: CI workflow

8. Evaluation Criteria

Your submission will be evaluated primarily on engineering quality and reasoning, not on the raw number of automated tests.

Area	Weight	What we assess
Test design & coverage	20%	Risk identification, meaningful scenarios, negative/edge coverage, prioritization.
Automation architecture	20%	Maintainability, abstraction, reuse, structure, configuration.
Functional automation	20%	Correctness, end-to-end value, reliable execution, useful assertions.
Negative & edge testing	10%	Validation of failure states, boundaries, data/state transitions.
Synchronization & stability	10%	Flake resistance, waits, deterministic behavior, CI thinking.
Code quality	5%	Readability, naming, simplicity, consistency.
Reporting & documentation	5%	README, reports, defects, troubleshooting, clarity.
Engineering judgment	10%	What you automate, what you do not, trade-offs, scaling/debugging approach.

Submission Checklist

- Test strategy completed.
- Detailed test cases included and prioritized.
- 15–20 high-value automated scenarios implemented.
- Required categories covered.
- Negative and asynchronous scenarios included.
- Assertions validate actual behavior/state.
- Failure evidence and reporting available.
- README allows another engineer to run the suite without hand-holding.
- Defects/findings documented, if any.
- Trade-off questions answered.